USEI13 – Rail Hub Centrality Analysis

Context

Rail operations rely on hub stations to manage transfers, connect regions, and handle high traffic volumes. Identifying these quantitatively helps:

- Capacity planning and scheduling
- Analyzing network robustness (identifying chokepoints)
- Prioritizing infrastructure upgrades

User Story

As a **maintenance planner**, I want to compute centrality measures for each station, such as **betweenness** (shortest-path broking), **harmonic closeness** (accessibility), and **strength/degree** (direct connectivity), and combine them into a **composite HubScore** (normalized and weighted), so I can rank and export the network's hubs for decision-making.

Acceptance Criteria

- Calculate **Degree**, **Strength**, **Betweenness**, and **Harmonic Closeness** for every station
- All scores must be **normalized to [0,1]**
- Calculate the **HubScore** using the formula:

HubScore = 0.35 · Betweenness_norm + 0.35 · HarmonicCloseness_norm + 0.30 · Strength_norm

- The output must be a **list of results ordered by HubScore (descending)**
- Include a **Temporal Complexity Analysis**

Temporal Complexity Analysis

1. Core Algorithm: All-Pairs Shortest Path

The overall time complexity is dominated by the computation of **Betweenness** and **Harmonic Closeness** centralities, both of which require shortest-path calculations between **all pairs of nodes** in an **undirected rail network**.

This is achieved by executing **Dijkstra's Algorithm |V| times**, once per station acting as the source node, using the **Brandes Algorithm** to efficiently accumulate centrality scores.

---

2. Detailed Complexity Analysis (Dijkstra Execution)

The complexity of a single execution of **Dijkstra's Single-Source Shortest Path (SSSP)** algorithm, assuming:

- A sparse graph representation
- A Min-Heap (Priority Queue)

is:

O_Dijkstra = O((V + E) log V)

---

3. Total Complexity Derivation

Because the algorithm is executed **|V| times** (as required by the Brandes methodology), the total computational complexity becomes:

O_Total = |V| × O_Dijkstra = O(V · (E + V log V))

Code Implementation Overview

The `computeShortestPathMeasures` method reflects the above complexity model, as it repeatedly executes modified Dijkstra searches for each station.

Algorithm Structure

- **Outer Loop**
  Iterates through all stations as source nodes:
  `for (RailwayStation s : vertices)` ⟹ **O(V)**: Calculating Betweenness Centrality (and Harmonic Proximity) requires that the shortest path algorithm (Dijkstra's algorithm) be run from each node in the graph. If the graph has |V| vertices (stations), the outer loop runs |V| times.

- **Inner Loop**
  Priority Queue–based Dijkstra execution:
  `while (!pq.isEmpty())` ⟹ **O(E + V log V)** per source: This complexity is typical of Dijkstra's algorithm for sparse graphs (where E is small) and derives directly from the efficiency of the Min-Heap (Priority Queue) structure in handling costs.

  The complexity is the sum of the two main costs of Dijkstra's algorithm:

  1. Cost of Vertices (V logV): For each of the |V| vertices, the most expensive operation is to remove it from the queue with the shortest distance (pq.poll().getKey()). In a Heap (Priority Queue), the removeMin() operation costs O(log V). The total is V × O(log V).
  2. Cost of Edges (E logV): The algorithm iterates over all |E| edges (graph.outgoingEdges(u)). If an edge relaxes its distance, the adjacent node has its value updated in the queue (or is reinserted). The update/insertion costs O(log V). The total is E × O(log V).

- **Backward Phase**
  Dependency accumulation for Betweenness Centrality:
  O(V + E) per source: After completing Dijkstra's shortest-path computation, the Brandes algorithm performs a **back-propagation phase** to compute the actual Betweenness dependency values (`δ_s(w)`).

  This phase has two components:

  - **Vertex ordering by distance**: vertices are sorted in descending distance order, with a cost of **O(V log V)**.
  - **Dependency propagation**: a linear traversal where each vertex and edge is visited once, costing **O(V + E)**.

  Although the sorting step costs **O(V log V)**, it is dominated by the earlier Dijkstra execution (**O((V + E) log V)**). Therefore, the effective complexity of the back-propagation step is **linear**, i.e.:

  O(V + E)

  This linear dependency accumulation is the key factor that makes the **Brandes algorithm** more efficient than naive Betweenness computation approaches.

Java Implementation Example

```
private void computeShortestPathMeasures(
        Graph<RailwayStation, Double> graph,
        Map<RailwayStation, HubCentralityResult> resultMap) {

    // V repetitions for all sources (O(V))
    for (RailwayStation s : vertices) {

        // --- Forward phase: Modified Dijkstra ---
        // PriorityQueue-based SSSP: O(E log V + V log V)
        while (!pq.isEmpty()) {
            // ... shortest path relaxation and path counting ...
        }

        // --- Backward phase: Betweenness accumulation ---
        // Complexity: O(V + E)
    }
}
```

Current Complexity

Because **shortest-path computations are executed independently for each station** , the algorithm runs a full **Dijkstra-based SSSP** for every vertex, even though large portions of the network structure are reused:

$O(V \cdot (E + V \log V))$

This dominates the overall execution time, as Betweenness and Harmonic Closeness both rely on APSP-style processing via Brandes' algorithm.

Optimal Complexity (theoretical lower bound for weighted graphs)

Using **Brandes' algorithm with Dijkstra and Min-Heap**, the asymptotically optimal complexity for computing Betweenness and Harmonic Closeness in a sparse, weighted graph is:

$O(V \cdot (E + V \log V))$

Further asymptotic improvement is not possible without:

- restricting graph structure,
- approximating centrality,
- or changing the problem formulation.

Summary Table

Operation	Expected Complexity	Worst Case
Degree & Strength	O(V + E)	O(V + E)
Betweenness Centrality	O(V · (E + V log V))	O(V · (E + V log V))
Harmonic Closeness Centrality	O(V · (E + V log V))	O(V · (E + V log V))
Normalization & HubScore Sorting	O(V log V)	O(V log V)
Full USEI13 Hub Analysis Pipeline	O(V · (E + V log V))	O(V · (E + V log V))